Zepto Review Discovery Engine – Technical Deep Dive

How 15,607 Zepto Play Store & Multi-Channel Reviews Become Grounded, Queryable Product Insight

This document explains what the engine is, how it works end to end, the reasoning behind each design decision, and the technical constraints that shaped it. It is written so an evaluator can verify the engineering—not just the narrative.

1 · The Problem This Engine Solves

Zepto's strategic goal is to increase cross-category product discovery and reduce low-margin repeat staple reliance. Before proposing a product solution, a PM needs to know what users actually struggle with—at scale, from real evidence, not anecdote. The raw material exists (tens of thousands of public reviews across App Store, Play Store, Reddit, YouTube, X) but is unusable by hand:

• **Volume**: 15,607 reviews in a 10-week window—no product team reads that manually.

• **Noise**: reviews mix delivery partner bugs, app crashes, billing issues, and discovery complaints in no order.

• **Trust**: any manual summary a human writes is hard to verify against the original customer source.

The engine turns that pile into 4 ranked discovery-friction themes, 6 behavioral customer segments, and a grounded chatbot that answers product questions, with every answer traceable back to specific review IDs.

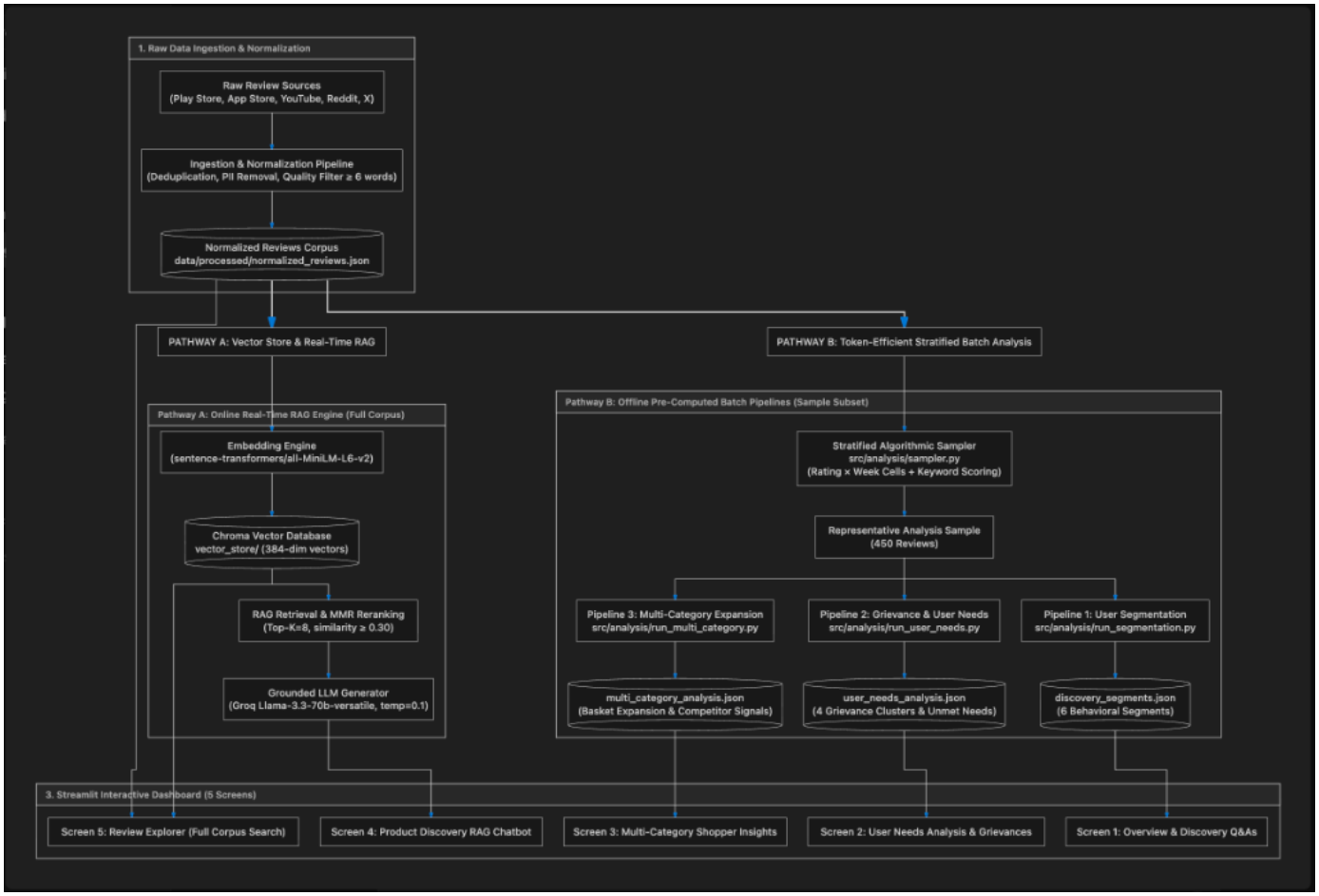
2 · Why a RAG Architecture (and Not the Alternatives)

Approach	Why Not
Read reviews manually / spreadsheet tagging	Weeks of effort; subjective; doesn't scale; zero reproducibility.
Dump all reviews into one giant LLM prompt	**Impossible**—15.6K reviews (~900K tokens) vastly exceed free-tier API rate limits (12K tokens/min). Also expensive and unauditable.
Pure keyword search / regex	Misses semantic intent—'received expired curd' and 'best before date missing' never match on exact regex keywords but are the same complaint.
RAG + Stratified Sampling (Chosen)	Embeds reviews by semantic meaning, retrieves only relevant reviews per query, and grounds the LLM's answer in real text—scalable, cheap, and auditable.

***Core principle — grounding**: the LLM is treated as untrusted until its output is tied back to real review_ids. This is what prevents hallucinated insights and makes the engine safe for a PM to act on.*

3 · End-to-End Workflow & Architecture Diagram

The system is architected into two parallel processing pathways (Offline Pre-Computed Batch Analysis vs. Online Real-Time RAG):



Stage-by-Stage Implementation Detail

Stage	What Happens	Tech / Decision
Ingest	Pull public reviews across 5 channels (Google Play, App Store, YouTube, Reddit, X), 10-week window.	google-play-scraper, PRAW, YouTube API (Plain Python codebase)
Normalize	Dedupe near-identical reviews; filter short noise (<6 words); strip PII (emails, numbers); unify schema.	Deterministic Python pipeline (data/processed/normalized_reviews.json)
Embed & Index	Convert each review to a 384-dim meaning vector; store in persistent vector store keyed by review_id.	Local sentence-transformers all-MiniLM-L6-v2 + Chroma DB
Analyze (Batch)	Sample representative 450 reviews across rating × week cells; execute 3 offline pipelines (Segmentation, Grievances, Multi-Category).	sampler.py + Groq batch processing (ANALYSIS_BATCH_SIZE=20)
Retrieve (RAG)	For ad-hoc questions, embed query and pull top matching reviews using Cosine distance + MMR reranking.	Chroma DB similarity search (top_k=8, candidate pool k=40, lambda=0.7)
Generate (RAG)	LLM writes grounded answer using ONLY retrieved reviews; cites review_ids; refuses off-scope.	Groq llama-3.3-70b-versatile (temperature=0.1)
Validate	Enforce theme caps, review_id provenance checks, and PII blocking before surfacing to UI.	Deterministic validator (validators.py & run_metadata.json)
Surface	5 interactive dashboard screens: Overview, User Needs, Multi-Category, RAG Chatbot, Review Explorer.	Streamlit app deployed to Community Cloud

4 · What 'Embedding' and 'RAG' Actually Mean (Plain Version)

Embedding converts each review into a list of 384 numbers (a vector) that captures its semantic meaning. Reviews with similar meaning end up close together in vector space, even if they share no words. That's why 'received expired curd' and 'best before date missing' cluster together automatically.

RAG (Retrieval-Augmented Generation) is a two-step pattern: retrieve the handful of reviews most relevant to a specific user question, then generate an answer using only those retrieved reviews. The LLM never answers from general internet knowledge—it answers strictly from retrieved evidence, which makes every claim citable.

5 · Why Local Embeddings, Groq for Generation

The architecture evaluates local models vs hosted endpoints. The system uses a local SentenceTransformer model paired with Groq Cloud API for generation:

Task	Engine Chosen	Why
Embedding 15,607 reviews + each query	Local MiniLM (384-dim)	Free, 100% offline, zero API token budget consumed, runs fast on CPU.
Theme analysis + chatbot answers	Groq llama-3.3-70b	Strong strategic reasoning; Groq's LPU speed makes responses interactive (<1s).

Net effect: zero embedding tokens consumed, so the entire scarce Groq token budget is reserved for generation.

6 · Why 450 Reviews in Analysis, Not All 15,607 (The Token-Limit Math)

This is the most important engineering constraint, and a common evaluator question. The limit is not the model's context window—llama-3.3-70b has a large 128K-token context window. The binding constraint is **Groq's free-tier throughput cap**:

The Rate Limit Numbers (Groq Free Tier):

• 12,000 tokens per minute (TPM)

• 100,000 tokens per day (TPD)

• 30 requests per minute (RPM), 1,000 requests/day

The math: a typical review is ~50–60 tokens. 15,607 reviews ≈ 850,000–950,000 input tokens. That is:
• ≈ **75× over the 12,000 tokens-per-minute ceiling**, and
• ≈ **9× over the 100,000 tokens-per-day cap**.

The fix — stratified sampling: Instead of passing all 15.6K reviews, sampler.py selects a 450-review representative sample, bucketed by rating tier × ISO week, deliberately oversampling negative reviews (1–2★) because that's where actionable discovery complaints concentrate. 450 reviews ≈ 22K tokens—comfortably runnable within limits. Crucially, **the chatbot still retrieves over all 15,607 reviews** in real-time.

7 · The Trust Layer — How Hallucination Is Prevented

LLMs can invent plausible-sounding claims. For a PM making roadmap bets, that's dangerous. The engine defends against it with a deterministic validation layer:

Check	Rule Enforced
Provenance	Every theme and chatbot claim must cite real review_id(s) present in the corpus.
Structural Integrity	≤4 friction clusters; off-topic noise routed to a general bucket.
Privacy	No usernames, emails, or phone numbers ever appear in UI output.
Scope Enforcement	Out-of-scope questions are refused rather than answered from general knowledge.

8 · Core Grievance Themes Surfaced

Theme	Reviews	Avg Rating	Key Signal
Search Failures & Irrelevant Recs	142	1.4★	Search typos fail; out-of-stock items boosted over available products.
Missing Product Specs & Expiry Dates	118	1.6★	Users hesitate to buy fresh/gourmet without visible expiry dates.
Out-of-Stock Disruption	105	1.8★	Frequent stockouts break habit loops and stop basket expansion.
Strict Refund & OTP Policy Friction	85	1.2★	Complex OTP/return flows on fresh items create high trial fear.

9 · Honest Limitations

• **Inferred segments**: user groups are estimated from review wording heuristics, not verified demographic data (app stores expose none).

• **Sample-based themes**: offline themes derive from a 450-review stratified sample; fixed seed (42) ensures exact reproducibility.

• **Ephemeral vector store**: on Streamlit Cloud, disk is ephemeral, so the vector index auto-rebuilds on cold start.

10 · Tech Stack Summary

Layer	Technology Choice
Ingestion	google-play-scraper, app-store-scraper, PRAW, YouTube API
Embeddings	sentence-transformers all-MiniLM-L6-v2 (local, 384-dim)
Vector Store	Chroma DB (file-persisted, keyed by review_id)
Generation	Groq llama-3.3-70b-versatile (temperature=0.1)
Dashboard	Streamlit (5 vertical navigation screens)
Live URL	https://zepto-review-discovery-engine-b5yqfnvwidxrqxxmffftsf.streamlit.app/